

Experiment 04 · I²C - Two-Wire Addressed Bus

I2C1 master and slave at 100 kHz, addressed transfer with hardware acknowledgement

NUCLEO-F446RE
× 2
STM32F446RET6,
Cortex-M4

I2C1 (APB1)
peripheral used

100 kHz standard mode
line settings

3 (SCL, SDA, GND) + pull-ups
wires between boards

Bare metal
CMSIS registers, no
HAL, no CubeMX

1 Aim and specification

Two NUCLEO-F446RE boards are made to talk to each other over **I2C** only. The board programmed as **master** reads its blue B1 user button and measures how long it is held; the board programmed as **slave** has no button and only reacts to what arrives on the I2C link, blinking its green LD2 LED in one of two clearly distinguishable patterns. After programming, both boards run from any 5 V USB supply with no PC attached.

1.1 Behavioural specification

Stimulus on the master	Measured press	Command transmitted	Slave LD2 pattern	LED-to-LED spacing	Total
Short click	< 1000 ms	0xA1 CMD_SHORT	3 blinks · 150 ms ON / 150 ms OFF	300 ms	0.9 s
Press and hold	≥ 1000 ms	0xA2 CMD_LONG	3 blinks · 1000 ms ON / 1000 ms OFF	2000 ms	6.0 s

The long-press pattern is the point of the experiment: 1000 ms on plus 1000 ms off means the slave LED starts a blink every **2.0 s**, against 300 ms for a short click. Both numbers are single `#define`s at the top of the slave listing (`LONG_ON_MS`, `LONG_OFF_MS`, `LONG_REPEAT`), so the timing can be retuned without touching the protocol code.

1.2 Status indication on the master

Indication	When	Meaning
Blue	steady while the frame is on the wire	transmission in progress
Green + one 250 ms LD2 flash	after a valid acknowledgement	the slave received the command
Red + three fast LD2 flashes	acknowledgement missing or wrong	wiring, power or protocol fault

The RGB LED is optional – set `USE_RGB` to 0 in the master listing and the on-board green LD2 alone carries the same information. If your RGB LED is common-anode, set `RGB_COMMON_CATHODE` to 0 instead of rewiring.

1.3 Frame format

All four experiments in this series use the same three-byte payload so that the protocols can be compared fairly:

Byte	Value	Purpose
1	0xAA	sync pattern – alternating bits, easy to recognise on an oscilloscope and unlikely to appear in noise
2	0xA1 or 0xA2	the command itself
3	~cmd (0x5E / 0x5D)	one's-complement checksum; the receiver acts only if it matches

2 Apparatus

Qty	Item	Notes
2	NUCLEO-F446RE development board	one programmed as master, one as slave
1 set	Male-male jumper wires (Dupont)	keep them under ~20 cm
2	USB Mini-B cables	for programming, then for standalone power
1-2	USB power bank or 5 V phone charger	to run the boards away from the PC
1	Common-cathode RGB LED + 3 × 330 Ω	optional master status indicator
1	Small breadboard	only needed for the RGB LED / pull-up resistors

3 Pin assignment

MCU pin	Function	Configuration	Morpho	Arduino	Used by
PB8	I2C1_SCL	AF4, open drain	CN10-3	D15	clock, master drives
PB9	I2C1_SDA	AF4, open drain	CN10-5	D14	bidirectional data
-	2 × 4.7 kΩ to +3V3	external resistors	CN7-16 (+3V3)	-	mandatory bus pull-ups
PC13	B1 user button	input, pull-up	CN7-23	-	master only
PA5	LD2 user LED	output, push-pull	CN10-11	D13	both boards
GND	0 V reference	-	CN10-20	-	mandatory

Reading the headers

CN7 and CN10 are the two 2×19 “morpho” headers along the long edges. Pin 1 of each is the row nearest the ST-LINK part of the board, odd numbers are in the left column and even numbers in the right column when the board is seen from above with the USB connector upwards. Every pin name is printed on the underside of the PCB – when in doubt, identify a pin by its *name*, not by counting.

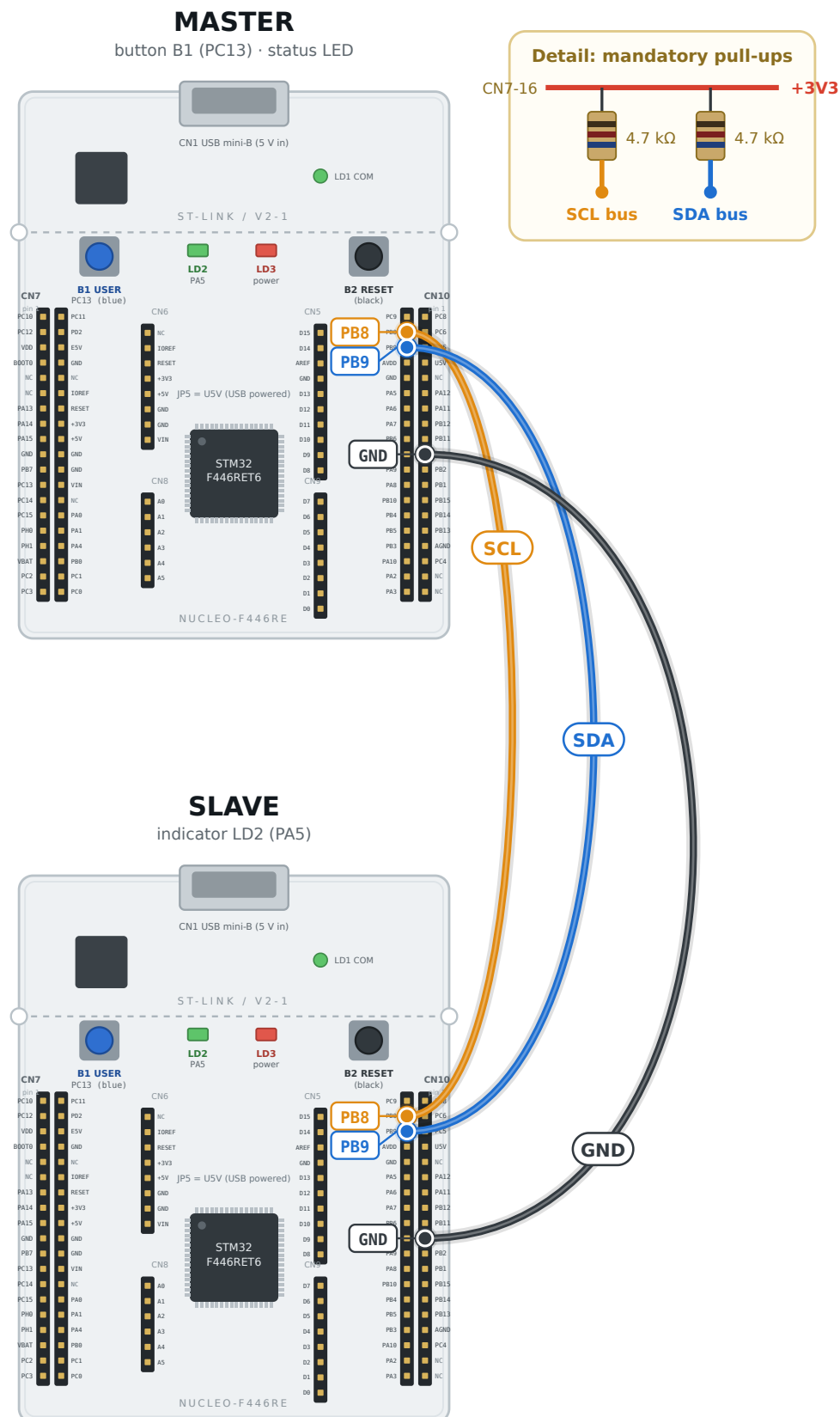
4 Circuit and wiring

Make every connection with the boards **unpowered**. The colours in the last column are the ones used in Figure 1 on the next page; any colour works electrically, but keeping to a convention makes a wiring fault obvious at a glance.

Signal	MASTER pin	SLAVE pin	Slave function	Wire colour used	Topology
SCL	PB8 · CN10-3	PB8 · CN10-3	SCL	orange	common bus
SDA	PB9 · CN10-5	PB9 · CN10-5	SDA	blue	common bus
GND	GND · CN10-20	GND · CN10-20	GND	black	common
Pull-ups	4.7 kΩ SCL → +3V3	4.7 kΩ SDA → +3V3	-	-	one pair for the whole bus

Experiment 4 · I2C1 - board-to-board wiring

Top view, USB connectors upwards. Pin names are silk-screened on the underside of each board.



SCL and SDA are common bus wires, not crossed. Both pins are open-drain, so the 4.7 kΩ pull-ups create the One resistor pair for the whole bus is enough - fit it on a small breadboard between the two boards.

PB8 = CN10-3 = D15 | PB9 = CN10-5 = D14 | $+3V3$ = CN7-16 | GND = CN10-20.

Figure 1 - Physical wiring between the two boards, drawn to the real board outline. The coloured rings mark the pads that carry a wire.

5 Theory of operation

Without the pull-up resistors nothing works

Both I2C pins are configured **open drain**: the output transistor can only pull the line down to 0 V, never up. The high level is produced entirely by the external resistors. The NUCLEO-F446RE has no pull-ups fitted on PB8/PB9, so two 4.7 kΩ resistors to +3V3 are part of the circuit, not an optional extra. The internal ~40 kΩ pull-ups enabled in the code are a weak fallback only, adequate for a few centimetres of wire at low speed.

I2C replaces the chip-select wire of SPI with an **address** sent over the bus itself. Every transfer starts with a START condition (SDA falls while SCL is high), followed by seven address bits and a direction bit. The addressed slave answers by pulling SDA low for one clock – the acknowledge – which is a hardware handshake that neither SPI nor UART has. The slave used here answers to address `0x28`.

Programming the bit rate

The classic F4 I2C peripheral needs three numbers instead of one baud-rate register:

Field	Value	Meaning
CR2.FREQ	16	the APB1 clock in MHz, so the peripheral can compute its own timings
CCR	80	standard mode: $CCR = f_{PCLK1} / (2 \times f_{SCL}) = 16\,000\,000 / 200\,000$
TRISE	17	maximum rise time 1000 ns expressed in PCLK1 periods, plus one: $1000/62.5 + 1$

The two STM32 quirks you must respect

- **Clearing ADDR.** The address-matched flag is cleared by reading `SR1` and then `SR2`, in that order. Reading `SR2` also tells the slave, through the `TRA` bit, whether the master wants to write or to read. Forgetting the second read leaves SCL stretched low and the bus frozen.
- **OAR1 bit 14 and the ACK bit.** Bit 14 of `OAR1` is reserved but must be written as 1. And `CR1.ACK` has to be set *after* `PE`, otherwise the enable sequence clears it again and the slave silently NACKs its own address.

The two-transaction protocol

The master first performs a **write** of `[0xAA][cmd][~cmd]` and closes it with a STOP. It then performs a separate one-byte **read**; for a single-byte read RM0390 requires that ACK is cleared before the ADDR flag and that STOP is requested immediately after, which is what `i2c1_read1()` does. The slave replies `0x5A` and only then blinks, so the bus is released long before the LED pattern finishes. Every wait in the master is bounded by a 25 ms timeout, so a missing slave produces a red indication instead of a hang.

5.1 Clock configuration used by both boards

Both listings begin with `clock_init_hsi_16mhz()`, which selects the internal 16 MHz RC oscillator, leaves every bus prescaler at /1 and sets zero flash wait states. Nothing is left to a start-up file or to CubeMX, so **SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz** and every timing number in this manual can be verified by hand. `SysTick` is then reloaded with 15999 to interrupt once per millisecond, which is the time base for both the button measurement and the blink pattern.

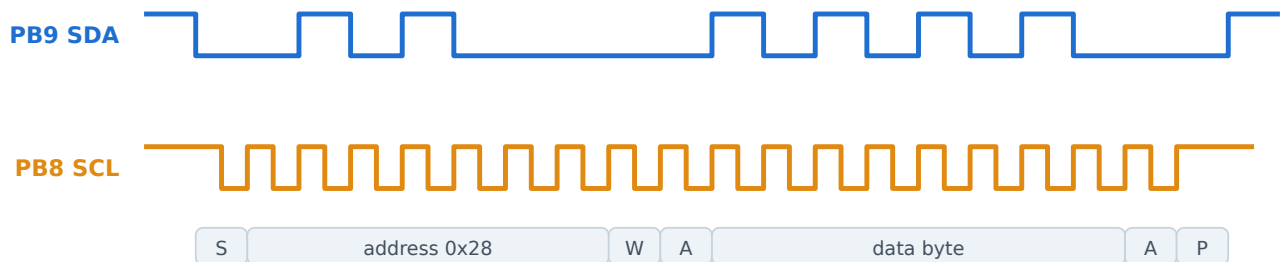
5.2 Register configuration summary

Register	Value written	Why
RCC->APB1ENR	<code> = I2C1EN</code>	clock to I2C1
GPIOB->OTYPER	PB8, PB9 = <code>1</code>	open drain – the defining setting of an I2C pin
GPIOB->AFR[1]	PB8, PB9 = <code>4</code>	AF4 = I2C1

I2C1->CR2	16	FREQ = APB1 clock in MHz
I2C1->CCR	80	100 kHz standard mode
I2C1->TRISE	17	1000 ns maximum rise time
I2C1->OAR1 (slave)	(0x28<<1) (1<<14)	own 7-bit address; bit 14 must be written as 1
I2C1->CR1	PE , then ACK	enable, then acknowledge – the order matters
I2C1->CR1	START / STOP	generate the start and stop conditions
I2C1->SR1	SB, ADDR, TXE, RXNE, BTF, AF, STOPF	start sent, address matched, data flow, byte finished, NACK, stop detected
I2C1->SR2	BUSY , TRA	bus occupied; transfer direction (read after SR1 to clear ADDR)

6 Signal timing

I2C1 standard mode - write transaction to slave address 0x28



S = START (SDA falls while SCL is high) · A = the addressed slave pulls SDA low to acknowledge · P = STOP.
Every high level on the bus is produced by the pull-up resistor, never by a transistor.

Figure 2 – A write transaction. Both devices can only pull the lines down; the rise back to 3.3 V is done by the pull-up resistors, which is why their value sets the maximum bus speed.

7 MASTER source code **i2c_master.c**

Paste this whole listing over the contents of `Core/Src/main.c` in the master project. It is self-contained: there are no other files to add and no header to edit.

i2c_master.c

```
1/*=====
2 * EXPERIMENT 4 : I2C (I2C1, STANDARD MODE 100 kHz)      --- M A S T E R ---
3 * Board : NUCLEO-F446RE (STM32F446RET6, Cortex-M4)
4 * Style : 100 % bare-metal CMSIS register programming - no HAL, no CubeMX.
5 *
6 * BUS SETTINGS
7 * PB8 = I2C1_SCL (AF4, Arduino D15) PB9 = I2C1_SDA (AF4, Arduino D14)
8 * Both pins are ALTERNATE FUNCTION + OPEN DRAIN - this is what makes the
9 * wired-AND bus possible. Pull-up resistors are mandatory:
10 * 4.7 kOhm from SCL to +3V3 and 4.7 kOhm from SDA to +3V3
11 * (the internal ~40 kOhm pull-ups are enabled as a fallback only).
12 *
13 * FREQ = 16 APB1 clock in MHz, tells the peripheral its input clk
14 * CCR = 80 Sm mode: CCR = fPCLK1 / (2 x fSCL) = 16e6/(2x100e3)
15 * TRISE = 17 max rise time 1000 ns / 62.5 ns + 1
16 *
17 * TRANSACTION (7-bit slave address 0x28)
18 * 1) WRITE [0xAA sync][cmd][~cmd] -> slave stores the command
19 * 2) READ 1 byte -> slave answers ACK = 0x5A
20 * 3) slave then plays the LED pattern; the bus is already free
21 *
22 * WIRING (bus - both signals are common, NOT crossed)
23 * PB8 SCL (CN10-3 / D15) <-> PB8 SCL (CN10-3 / D15) + 4k7 to 3V3
24 * PB9 SDA (CN10-5 / D14) <-> PB9 SDA (CN10-5 / D14) + 4k7 to 3V3
25 * GND (CN10-20) <-> GND (CN10-20)
26 * 3V3 for the pull-ups may come from either board (CN7-16).
27 *
28 * Optional RGB status LED on the MASTER (common cathode + 3 x 330 R):
29 * PC0 = RED (CN7-38), PC1 = GREEN (CN7-36), PC2 = BLUE (CN7-35).
30 *=====*/
31
32#include "stm32f446xx.h"
33#include <stdint.h>
34
35/*----- user configuration -----*/
36#define USE_RGB 1u
37#define RGB_COMMON_CATHODE 1u
38
39#define LED_PIN 5u /* PA5 - LD2 green user LED */
40#define BTN_PIN 13u /* PC13 - B1 blue user button */
41
42#define SLAVE_ADDR7 0x28u /* 7-bit address of the slave board */
43
44#define CMD_SYNC 0xAAu
45#define CMD_SHORT 0xA1u
46#define CMD_LONG 0xA2u
47#define ACK_BYTE 0x5Au
48
49#define T_DEBOUNCE_MS 20u
50#define T_LONG_MS 1000u
51#define I2C_TIMEOUT_MS 25u
52
53/*===== 1. CLOCK : HSI 16 MHz, no PLL =====*/
54static void clock_init_hsi_16mhz(void)
55{
56 RCC->CR |= RCC_CR_HSION;
57 while ((RCC->CR & RCC_CR_HSIRDY) == 0u) { }
58
59 FLASH->ACR = FLASH_ACR_LATENCY_0WS;
60
61 RCC->CFGR &= ~RCC_CFGR_SW;
62 while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) { }
63
```

```

64   RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
65}
66
67/*===== 2. SysTick 1 ms time base =====*/
68static volatile uint32_t s_ticks;
69
70void SysTick_Handler(void) { s_ticks++; }
71
72static void systick_init(void)
73{
74   SysTick->LOAD = 16000u - 1u;
75   SysTick->VAL  = 0u;
76   SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
77                  SysTick_CTRL_TICKINT_Msk  |
78                  SysTick_CTRL_ENABLE_Msk;
79}
80static uint32_t millis(void) { return s_ticks; }
81static void delay_ms(uint32_t ms)
82{
83   uint32_t t0 = s_ticks;
84   while ((uint32_t)(s_ticks - t0) < ms) { __NOP(); }
85}
86
87/*===== 3. LD2 and B1 =====*/
88static void led_init(void)
89{
90   RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
91   (void)RCC->AHB1ENR;
92
93   GPIOA->MODER &= ~(3u << (LED_PIN * 2u));
94   GPIOA->MODER |= (1u << (LED_PIN * 2u));
95   GPIOA->OTYPER &= ~(1u << LED_PIN);
96   GPIOA->OSPEEDR &= ~(3u << (LED_PIN * 2u));
97   GPIOA->PUPDR &= ~(3u << (LED_PIN * 2u));
98   GPIOA->BSRR  = (1u << (LED_PIN + 16u));
99}
100static void led_on (void) { GPIOA->BSRR = (1u << LED_PIN); }
101static void led_off(void) { GPIOA->BSRR = (1u << (LED_PIN + 16u)); }
102
103static void button_init(void)
104{
105   RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
106   (void)RCC->AHB1ENR;
107
108   GPIOC->MODER &= ~(3u << (BTN_PIN * 2u));
109   GPIOC->PUPDR &= ~(3u << (BTN_PIN * 2u));
110   GPIOC->PUPDR |= (1u << (BTN_PIN * 2u));
111}
112static uint32_t button_down(void)
113{
114   return ((GPIOC->IDR & (1u << BTN_PIN)) == 0u);
115}
116static uint8_t button_capture(uint32_t *press_ms)
117{
118   uint32_t t0, d;
119
120   if (!button_down()) return 0u;
121   delay_ms(T_DEBOUNCE_MS);
122   if (!button_down()) return 0u;
123
124   t0 = millis();
125   while (button_down()) { __NOP(); }
126   delay_ms(T_DEBOUNCE_MS);
127
128   d = millis() - t0;
129   if (press_ms != 0) { *press_ms = d; }
130   return (d >= T_LONG_MS) ? CMD_LONG : CMD_SHORT;
131}
132
133/*===== 4. Optional RGB indicator =====*/
134static void rgb_init(void)
135{

```

```

136 #if USE_RGB
137     uint32_t p;
138     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
139     (void)RCC->AHB1ENR;
140     for (p = 0u; p <= 2u; p++) {
141         GPIOC->MODER &= ~(3u << (p * 2u));
142         GPIOC->MODER |= (1u << (p * 2u));
143         GPIOC->OTYPER &= ~(1u << p);
144         GPIOC->PUPDR &= ~(3u << (p * 2u));
145     }
146 #endif
147 }
148 static void rgb_set(uint32_t r, uint32_t g, uint32_t b)
149 {
150     #if USE_RGB
151     #if RGB_COMMON_CATHODE
152         GPIOC->BSRR = (r ? (1u << 0) : (1u << 16)) |
153             (g ? (1u << 1) : (1u << 17)) |
154             (b ? (1u << 2) : (1u << 18));
155     #else
156         GPIOC->BSRR = (r ? (1u << 16) : (1u << 0)) |
157             (g ? (1u << 17) : (1u << 1)) |
158             (b ? (1u << 18) : (1u << 2));
159     #endif
160 #else
161     (void)r; (void)g; (void)b;
162 #endif
163 }
164 static void master_indicate(uint32_t ok)
165 {
166     uint32_t i;
167     if (ok) {
168         rgb_set(0u, 1u, 0u);
169         led_on(); delay_ms(250u); led_off();
170     } else {
171         rgb_set(1u, 0u, 0u);
172         for (i = 0u; i < 3u; i++) { led_on(); delay_ms(60u); led_off(); delay_ms(60u); }
173     }
174     rgb_set(0u, 0u, 0u);
175 }
176
177 /*===== 5. I2C1 in MASTER mode =====*/
178 static void i2c1_master_init(void)
179 {
180     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
181     RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;
182     (void)RCC->APB1ENR;
183
184     GPIOB->MODER &= ~(3u << (8u*2u)) | (3u << (9u*2u));
185     GPIOB->MODER |= ((2u << (8u*2u)) | (2u << (9u*2u))); /* I0 = AF */
186     GPIOB->OTYPER |= ((1u << 8u) | (1u << 9u)); /* OPEN DRAIN */
187     GPIOB->OSPEEDR |= ((3u << (8u*2u)) | (3u << (9u*2u)));
188     GPIOB->PUPDR &= ~(3u << (8u*2u)) | (3u << (9u*2u));
189     GPIOB->PUPDR |= ((1u << (8u*2u)) | (1u << (9u*2u))); /* weak pull-up */
190
191     GPIOB->AFR[1] &= ~(0xFu << ((8u-8u)*4u)) | (0xFu << ((9u-8u)*4u));
192     GPIOB->AFR[1] |= ((4u << ((8u-8u)*4u)) | (4u << ((9u-8u)*4u))); /* AF4 */
193
194     /* Reset the peripheral so a half-finished transfer cannot lock the bus. */
195     RCC->APB1RSTR |= RCC_APB1RSTR_I2C1RST;
196     RCC->APB1RSTR &= ~RCC_APB1RSTR_I2C1RST;
197
198     I2C1->CR1 = 0u;
199     I2C1->CR2 = 16u; /* FREQ[5:0] = APB1 clock in MHz */
200     I2C1->CCR = 80u; /* Sm, 100 kHz */
201     I2C1->TRISE = 17u; /* 1000 ns max rise time */
202     I2C1->CR1 |= I2C_CR1_PE;
203 }
204
205 static uint32_t i2c_wait(uint32_t mask, uint32_t ms)
206 {
207     uint32_t t0 = millis();

```



```

208 while ((I2C1->SR1 & mask) == 0u) {
209     if (I2C1->SR1 & I2C_SR1_AF) { I2C1->SR1 &= ~I2C_SR1_AF; return 0u; }
210     if ((uint32_t)(millis() - t0) > ms) { return 0u; }
211 }
212 return 1u;
213}
214static void i2c_abort(void)
215{
216     I2C1->CR1 |= I2C_CR1_STOP;
217     I2C1->SR1 = 0u;          /* clear AF / BERR / ARLO */
218}
219
220/* Master transmitter: START - address+W - n data bytes - STOP. */
221static uint32_t i2c1_write(uint8_t addr7, const uint8_t *data, uint32_t n)
222{
223     uint32_t i, t0 = millis();
224
225     while (I2C1->SR2 & I2C_SR2_BUSY) {
226         if ((uint32_t)(millis() - t0) > I2C_TIMEOUT_MS) { return 0u; }
227     }
228
229     I2C1->CR1 |= I2C_CR1_START;
230     if (!i2c_wait(I2C_SR1_SB, I2C_TIMEOUT_MS)) { i2c_abort(); return 0u; }
231     (void)I2C1->SR1;
232     I2C1->DR = (uint32_t)(addr7 << 1u);          /* R/W = 0 */
233
234     if (!i2c_wait(I2C_SR1_ADDR, I2C_TIMEOUT_MS)) { i2c_abort(); return 0u; }
235     (void)I2C1->SR1; (void)I2C1->SR2;          /* clear ADDR */
236
237     for (i = 0u; i < n; i++) {
238         if (!i2c_wait(I2C_SR1_TXE, I2C_TIMEOUT_MS)) { i2c_abort(); return 0u; }
239         I2C1->DR = (uint32_t)data[i];
240     }
241     if (!i2c_wait(I2C_SR1_BTF, I2C_TIMEOUT_MS)) { i2c_abort(); return 0u; }
242
243     I2C1->CR1 |= I2C_CR1_STOP;
244     return 1u;
245}
246
247/* Master receiver, single byte (RM0390 "one byte" procedure). */
248static uint32_t i2c1_read1(uint8_t addr7, uint8_t *b)
249{
250     uint32_t t0 = millis();
251
252     while (I2C1->SR2 & I2C_SR2_BUSY) {
253         if ((uint32_t)(millis() - t0) > I2C_TIMEOUT_MS) { return 0u; }
254     }
255
256     I2C1->CR1 &= ~I2C_CR1_ACK;          /* NACK the only byte */
257     I2C1->CR1 |= I2C_CR1_START;
258     if (!i2c_wait(I2C_SR1_SB, I2C_TIMEOUT_MS)) { i2c_abort(); return 0u; }
259     (void)I2C1->SR1;
260     I2C1->DR = (uint32_t)((addr7 << 1u) | 1u);          /* R/W = 1 */
261
262     if (!i2c_wait(I2C_SR1_ADDR, I2C_TIMEOUT_MS)) { i2c_abort(); return 0u; }
263     (void)I2C1->SR1; (void)I2C1->SR2;          /* clear ADDR */
264     I2C1->CR1 |= I2C_CR1_STOP;          /* STOP right after ADDR */
265
266     if (!i2c_wait(I2C_SR1_RXNE, I2C_TIMEOUT_MS)) { i2c_abort(); return 0u; }
267     *b = (uint8_t)I2C1->DR;
268     return 1u;
269}
270
271/*===== main =====*/
272int main(void)
273{
274     uint8_t cmd, ack, frame[3];
275     uint32_t press_ms, ok;
276
277     clock_init_hsi_16mhz();
278     systick_init();
279     led_init();

```

```

280 button_init();
281 rgb_init();
282 i2c1_master_init();
283
284 rgb_set(0u, 0u, 1u); led_on(); delay_ms(300u); led_off(); rgb_set(0u,0u,0u);
285
286 for (;;) {
287     press_ms = 0u;
288     cmd = button_capture(&press_ms);
289     if (cmd == 0u) { continue; }
290
291     frame[0] = CMD_SYNC;
292     frame[1] = cmd;
293     frame[2] = (uint8_t)(cmd ^ 0xFFu);
294
295     rgb_set(0u, 0u, 1u);
296
297     ok = i2c1_write(SLAVE_ADDR7, frame, 3u);
298     ack = 0u;
299     if (ok) {
300         delay_ms(2u); /* let the slave re-arm */
301         ok = i2c1_read1(SLAVE_ADDR7, &ack) && (ack == ACK_BYTE);
302     }
303
304     rgb_set(0u, 0u, 0u);
305     master_indicate(ok);
306 }
307}

```

8 SLAVE source code `i2c_slave.c`

Paste this whole listing over the contents of `Core/Src/main.c` in the slave project. It is self-contained: there are no other files to add and no header to edit.

`i2c_slave.c`

```
1/*=====
2 * EXPERIMENT 4 : I2C (I2C1, STANDARD MODE 100 kHz)      --- S L A V E ---
3 * Board : NUCLEO-F446RE (STM32F446RET6, Cortex-M4)
4 * Style : 100 % bare-metal CMSIS register programming - no HAL, no CubeMX.
5 *
6 * SLAVE SPECIFICS
7 * OAR1 holds the 7-bit own address in bits [7:1]. Bit 14 of OAR1 must
8 * always be written to 1 (reserved, see RM0390) or address matching is
9 * unreliable. ACK (CR1.ACK) must be set AFTER PE = 1, otherwise it is
10 * cleared again by the peripheral enable sequence.
11 * A slave never programs CCR - the master supplies SCL.
12 *
13 * Address matched      -> SR1.ADDR = 1
14 * Direction of the frame -> SR2.TRA (0 = we receive, 1 = we transmit)
15 * ADDR is cleared by reading SR1 and then SR2 (the classic STM32 dance)
16 * STOPF is cleared by reading SR1 and then WRITING to CR1
17 *
18 * WIRING
19 * PB8 SCL (CN10-3 / D15) <-> master PB8 + 4k7 pull-up to 3V3
20 * PB9 SDA (CN10-5 / D14) <-> master PB9 + 4k7 pull-up to 3V3
21 * GND (CN10-20) <-> master GND
22 *
23 * BLINK SPECIFICATION
24 * CMD_SHORT (0xA1) -> 3 blinks, 150 ms ON / 150 ms OFF (300 ms spacing)
25 * CMD_LONG (0xA2) -> 3 blinks, 1000 ms ON / 1000 ms OFF (2 s spacing)
26 *=====*/
27
28#include "stm32f446xx.h"
29#include <stdint.h>
30
31#define LED_PIN          5u          /* PA5 - LD2 green user LED */
32#define OWN_ADDR7        0x28u      /* must match SLAVE_ADDR7 in the master */
33
34#define CMD_SYNC          0xAAu
35#define CMD_SHORT         0xA1u
36#define CMD_LONG          0xA2u
37#define ACK_BYTE          0x5Au
38
39#define SHORT_ON_MS       150u
40#define SHORT_OFF_MS      150u
41#define SHORT_REPEAT      3u
42#define LONG_ON_MS        1000u
43#define LONG_OFF_MS       1000u
44#define LONG_REPEAT       3u
45
46#define EVT_TIMEOUT_MS    50u
47
48/*===== 1. CLOCK : HSI 16 MHz, no PLL =====*/
49static void clock_init_hsi_16mhz(void)
50{
51    RCC->CR |= RCC_CR_HSION;
52    while ((RCC->CR & RCC_CR_HSIRDY) == 0u) { }
53
54    FLASH->ACR = FLASH_ACR_LATENCY_0WS;
55
56    RCC->CFGR &= ~RCC_CFGR_SW;
57    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) { }
58
59    RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
60}
61
62/*===== 2. SysTick 1 ms time base =====*/
63static volatile uint32_t s_ticks;
```

```

64
65void SysTick_Handler(void) { s_ticks++; }
66
67static void systick_init(void)
68{
69    SysTick->LOAD = 16000u - 1u;
70    SysTick->VAL = 0u;
71    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
72                  SysTick_CTRL_TICKINT_Msk |
73                  SysTick_CTRL_ENABLE_Msk;
74}
75static uint32_t millis(void) { return s_ticks; }
76static void delay_ms(uint32_t ms)
77{
78    uint32_t t0 = s_ticks;
79    while ((uint32_t)(s_ticks - t0) < ms) { __NOP(); }
80}
81
82/*===== 3. On-board LD2 =====*/
83static void led_init(void)
84{
85    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
86    (void)RCC->AHB1ENR;
87
88    GPIOA->MODER &= ~(3u << (LED_PIN * 2u));
89    GPIOA->MODER |= (1u << (LED_PIN * 2u));
90    GPIOA->OTYPER &= ~(1u << LED_PIN);
91    GPIOA->OSPEEDR &= ~(3u << (LED_PIN * 2u));
92    GPIOA->PUPDR &= ~(3u << (LED_PIN * 2u));
93    GPIOA->BSRR = (1u << (LED_PIN + 16u));
94}
95static void led_on (void) { GPIOA->BSRR = (1u << LED_PIN); }
96static void led_off(void) { GPIOA->BSRR = (1u << (LED_PIN + 16u)); }
97
98static void run_pattern(uint8_t cmd)
99{
100    uint32_t i, n, on, off;
101
102    if (cmd == CMD_LONG) { n = LONG_REPEAT; on = LONG_ON_MS; off = LONG_OFF_MS; }
103    else { n = SHORT_REPEAT; on = SHORT_ON_MS; off = SHORT_OFF_MS; }
104
105    for (i = 0u; i < n; i++) {
106        led_on(); delay_ms(on);
107        led_off(); delay_ms(off);
108    }
109}
110
111/*===== 4. I2C1 in SLAVE mode =====*/
112static void i2c1_slave_init(uint8_t addr7)
113{
114    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
115    RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;
116    (void)RCC->APB1ENR;
117
118    GPIOB->MODER &= ~((3u << (8u*2u)) | (3u << (9u*2u)));
119    GPIOB->MODER |= ((2u << (8u*2u)) | (2u << (9u*2u)));
120    GPIOB->OTYPER |= ((1u << 8u) | (1u << 9u)); /* OPEN DRAIN */
121    GPIOB->OSPEEDR |= ((3u << (8u*2u)) | (3u << (9u*2u)));
122    GPIOB->PUPDR &= ~((3u << (8u*2u)) | (3u << (9u*2u)));
123    GPIOB->PUPDR |= ((1u << (8u*2u)) | (1u << (9u*2u)));
124
125    GPIOB->AFR[1] &= ~((0xFu << ((8u-8u)*4u)) | (0xFu << ((9u-8u)*4u)));
126    GPIOB->AFR[1] |= ((4u << ((8u-8u)*4u)) | (4u << ((9u-8u)*4u)));
127
128    RCC->APB1RSTR |= RCC_APB1RSTR_I2C1RST;
129    RCC->APB1RSTR &= ~RCC_APB1RSTR_I2C1RST;
130
131    I2C1->CR1 = 0u;
132    I2C1->CR2 = 16u; /* FREQ = 16 MHz */
133    I2C1->OAR1 = ((uint32_t)addr7 << 1u) | (1u << 14u); /* bit14 must be 1 */
134    I2C1->OAR2 = 0u;
135    I2C1->CCR = 80u; /* harmless in slave mode */

```

```

136 I2C1->TRISE = 17u;
137 I2C1->CR1 |= I2C_CR1_PE;
138 I2C1->CR1 |= I2C_CR1_ACK; /* ACK only *after* PE = 1 */
139}
140
141/*===== main =====*/
142int main(void)
143{
144    uint8_t buf[4], cmd = 0u, valid = 0u;
145    uint32_t sr2, n, t0;
146
147    clock_init_hsi_16mhz();
148    systick_init();
149    led_init();
150    i2c1_slave_init(OWN_ADDR7);
151
152    led_on(); delay_ms(300u); led_off(); /* "slave alive" signal */
153
154    for (;;) {
155        if ((I2C1->SR1 & I2C_SR1_ADDR) == 0u) { continue; }
156
157        (void)I2C1->SR1;
158        sr2 = I2C1->SR2; /* reading SR2 clears ADDR */
159
160        if (sr2 & I2C_SR2_TRA) {
161            /*----- master is READING from us -----*/
162            t0 = millis();
163            while ((I2C1->SR1 & I2C_SR1_TXE) == 0u) {
164                if ((uint32_t)(millis() - t0) > EVT_TIMEOUT_MS) { break; }
165            }
166            I2C1->DR = valid ? (uint32_t)ACK_BYTE : 0x00u;
167
168            /* the master NACKs the single byte -> AF is set; clear it */
169            t0 = millis();
170            while ((I2C1->SR1 & I2C_SR1_AF) == 0u) {
171                if ((uint32_t)(millis() - t0) > EVT_TIMEOUT_MS) { break; }
172            }
173            I2C1->SR1 &= ~I2C_SR1_AF;
174            I2C1->CR1 |= I2C_CR1_ACK; /* re-arm for next frame */
175
176            if (valid) {
177                valid = 0u;
178                run_pattern(cmd); /* bus is free again now */
179            }
180        } else {
181            /*----- master is WRITING to us -----*/
182            n = 0u;
183            t0 = millis();
184            for (;;) {
185                uint32_t sr1 = I2C1->SR1;
186
187                if (sr1 & I2C_SR1_RXNE) {
188                    uint8_t b = (uint8_t)I2C1->DR;
189                    if (n < 4u) { buf[n] = b; }
190                    n++;
191                    t0 = millis();
192                } else if (sr1 & I2C_SR1_STOPF) {
193                    (void)I2C1->SR1;
194                    I2C1->CR1 |= I2C_CR1_PE; /* write to CR1 clears STOPF */
195                    break;
196                } else if ((uint32_t)(millis() - t0) > EVT_TIMEOUT_MS) {
197                    break;
198                }
199            }
200
201            if (n >= 3u && buf[0] == CMD_SYNC &&
202                (uint8_t)(buf[1] ^ 0xFFu) == buf[2] &&
203                (buf[1] == CMD_SHORT || buf[1] == CMD_LONG)) {
204                cmd = buf[1];
205                valid = 1u; /* answer on the next read */
206            }
207            I2C1->CR1 |= I2C_CR1_ACK;

```

```
208     }  
209 }  
210 }
```

9 Building it in STM32CubeIDE without touching CubeMX

1. **File → New → STM32 Project.** In the target selector open the *Board Selector* tab, type `NUCLEO-F446RE` and select it (or use the *MCU/MPU Selector* and pick `STM32F446RET6`).
2. Give the project a name, keep *Targeted Language* = **C** and *Targeted Binary Type* = **Executable**, and – this is the important one – set **Targeted Project Type = Empty**. An *Empty* project gives you the startup assembly file, the linker script, `system_stm32f4xx.c` and the CMSIS register headers, but **no HAL, no LL drivers and no .ioc file**. There is therefore nothing to configure graphically – which is exactly what a bare-metal experiment requires.
3. Answer **No** if the IDE offers to initialise peripherals; with an *Empty* project that dialog normally does not appear at all.
4. Open `Core/Src/main.c`, delete its whole contents and paste the listing from this manual. The single include `#include "stm32f446xx.h"` resolves through `Drivers/CMSIS/Device/ST/STM32F4xx/Include`, which the Empty project already put on the include path.
5. The symbol `STM32F446xx` is predefined by the project wizard. If the compiler complains that the header cannot decide which device it is, add it manually under *Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor*.
6. Build with **Ctrl + B**. Connect the first board, then *Run → Run As → STM32 C/C++ Application* to flash it.
7. Repeat steps 1–6 in a second project for the other listing, and flash it to the second board. Label the boards **MASTER** and **SLAVE** with a marker – they look identical once they are off the PC.

Alternative without the IDE: copy the generated `.bin` onto the `NODE_F446RE` mass-storage drive that appears when the board is plugged in, or use STM32CubeProgrammer.

10 Running the pair with no PC attached

Nothing in these programs depends on the debugger. The moment the board is powered, the reset vector in flash runs `main()`, so the two boards work as a stand-alone pair.

Option A - one USB supply per board (recommended)

1. Unplug both boards from the PC after flashing.
2. Plug each board's CN1 Mini-B socket into a USB power bank, a 5 V phone charger or a powered USB hub. Leave jumper **JP5 on U5V** (its factory position).
3. LD3 (red) lights up on each board and both LD2s give a single ~300 ms flash: that is the power-up self-test built into the firmware.
4. Keep the protocol wires and the common ground connected exactly as during programming.

Option B - one 5 V source for both boards

1. Move **JP5 to E5V** on *both* boards.
2. Feed +5 V into `E5V` (CN7 pin 6) and 0 V into `GND` (CN7 pin 8 or CN10 pin 20) of both boards from the same supply.
3. The grounds are then already common, so the separate GND jumper of the protocol wiring becomes redundant – but leaving it in place does no harm.

Safety

Never drive 5 V into the `+5V` pin while JP5 is on U5V and a USB cable is plugged in – you would connect two supplies together. Choose one powering scheme and stay with it. BOOT0 must remain low (its default) so the MCU boots from flash.

11 Test procedure and observations

1. Flash the master listing to board 1 and the slave listing to board 2.
2. Power both boards down, then wire them exactly as in the connection table. Check the ground wire twice – it is the single most common cause of a dead link.
3. Power both boards. Each LD2 must give one short flash: both CPUs are alive.
4. **Test 1 – short click.** Press and release B1 on the master quickly (< 1 s). The master should flash green/LD2 and the slave should give three fast blinks.
5. **Test 2 – long press.** Hold B1 for at least one second, then release. The slave must now blink three times with one second on and one second off, i.e. two seconds between the start of one blink and the start of the next.
6. Time the slow pattern with a stopwatch and record it. Repeat each test three times.
7. **Test 3 – error case.** Pull the data wire out and press the button again. The master must show the red / triple-flash failure indication instead of green.
8. If an oscilloscope or logic analyser is available, capture the line(s) during one press and compare the capture with the timing figure in section 6.

Trial	Press duration (ms)	Expected command	Master indication	Blinks counted	Measured spacing (s)	Pass
1						
2						
3						
4						
5						
6						

12 Troubleshooting

Symptom	Cause and remedy
The master always shows red, a scope shows SCL stuck low	The pull-up resistors are missing, or the ADDR flag was never cleared and the peripheral is stretching the clock. Reset both boards after fitting the resistors.
No acknowledge from the slave	The addresses do not match: <code>SLAVE_ADDR7</code> in the master must equal <code>OWN_ADDR7</code> in the slave, and <code>OAR1</code> must have bit 14 set.
It works once and then the bus is dead	A transfer was interrupted mid-byte. Both listings reset the peripheral through <code>APB1RSTR</code> at start-up, so pressing B2 RESET on both boards recovers it.
SDA and SCL swapped	PB8 is SCL and PB9 is SDA. Swapping them gives no acknowledge at all.
Neither LD2 flashes at power-up	The board is not running: check LD3 (red) is lit, that the USB supply really delivers power, and that you flashed the board you think you flashed.
The slave never reacts, master always shows red	Ground is not common. Join GND (CN10-20) of the two boards.
Behaviour is erratic, blinks appear at random	Wires too long or badly seated. Use jumpers under 20 cm and push them fully home.
Both boards blink but the patterns are swapped	The master listing was flashed to the slave board. Label the boards.
Long press is detected as a short one	You released before 1000 ms. The threshold is <code>T_LONG_MS</code> in the master listing – change it there if your lab asks for a different value.

13 Review questions

1. Why must I2C pins be open drain, and what does that imply for the pull-ups?
2. How is a START condition different from a normal data transition?
3. How does a slave acknowledge, and what happens if it does not?
4. Compute CCR and TRISE for 400 kHz fast mode with PCLK1 = 16 MHz.
5. What is clock stretching and which side does it?
6. Why must SR1 and SR2 be read in that order to clear ADDR?
7. How many devices can share an I2C bus and what actually limits the number?
8. Compare the wire count and throughput of I2C with SPI for the same task.